

Department of Computer Information Systems Engineering
CS-324 Machine Learning

FINAL PROJECT REPORT

**End-to-End Supervised Machine Learning System for
Intrusion Detection**

Project Title	End-to-End Supervised Machine Learning System for Intrusion Detection
Group Members	Amna Ahmed (CS-23016) Waleed Ahmed Khan (CS-23044) Abdul Wahab (CS-23035)
Instructor Name	Dr. Maria Waqas / Miss Mahnoor Malik
Submission Date	May 2026

Abstract

This project presents a complete supervised machine learning system for network intrusion detection. The objective is to classify flow-based network traffic as Normal or Malicious using engineered traffic features extracted from packet captures. The system follows an end-to-end pipeline that begins with controlled network traffic collection, continues through packet extraction and preprocessing, and ends with model training, evaluation, comparison, and an interactive prediction interface.

The dataset was generated from PCAP/PCAPNG traffic captures collected in a controlled virtual lab. Normal traffic included Google Meet/WebRTC, HTTPS/TLS, QUIC/HTTP3, browsing, and streaming patterns. Malicious traffic was simulated using controlled flood and scanning scenarios, including SYN flood, UDP flood, ICMP flood, TCP SYN scan, UDP scan, and aggressive scan activity. After time-window aggregation and feature engineering, the final dataset contained 11,051 samples and 25 predictor features plus one target label. The raw class distribution was near-balanced, with 5,211 Normal samples and 5,840 Malicious samples. The final implementation notebook further cleaned the dataset by dropping leakage-prone and highly correlated columns, leaving 9 predictor features for model training.

Three families of supervised algorithms were implemented: Logistic Regression, Decision Tree, and Neural Network. Each family was tested using three variants, and each variant was evaluated under two train/validation/test splits: 70/15/15 and 80/10/10. This produced 18 total trained models. Logistic Regression variants included L1, L2, and ElasticNet regularization. Decision Tree variants included shallow, balanced, and deep configurations. Neural Network variants included conservative, balanced, and aggressive architectures with dropout, regularization, and early stopping.

The models were evaluated using Accuracy, Precision, Recall, F1-score, ROC-AUC, False Positive Rate, and False Negative Rate. The best-performing model was NN-Aggressive under the 80/10/10 split. It achieved 0.9033 test accuracy, 0.8933 precision, 0.9257 recall, 0.9092 F1-score, 0.9377 ROC-AUC, 0.1214 FPR, and 0.0743 FNR. Overall, Neural Networks achieved the strongest average performance, Logistic Regression showed excellent stability, and the unconstrained Decision Tree demonstrated clear overfitting. The findings confirm that supervised ML can identify malicious network behavior effectively when packet traffic is converted into meaningful flow-level features.

Table of Contents

Section	Page
Abstract	2
1. Introduction	4
2. Dataset Collection and Preprocessing	4
3. Machine Learning Models	9
4. Training and Evaluation	12
5. Results and Comparative Analysis	12
6. Overfitting and Generalization Analysis	18
7. Interactive Notebook Interface	19
8. Best Model Discussion	20
9. Limitations	21
10. Future Improvements	21
11. Conclusion	21
12. References	22
13. Generative AI Use Declaration	22
Appendix	22

1. Introduction

1.1 Background

Cybersecurity focuses on protecting computer systems, networks, services, and data from unauthorized access, misuse, disruption, and damage. As networks grow in size and complexity, malicious activities such as scanning, flooding, denial-of-service behavior, and abnormal protocol usage become harder to detect through manual inspection alone.

An Intrusion Detection System (IDS) monitors traffic or host activity and raises an alert when suspicious behavior is detected. Traditional IDS solutions often depend on fixed signatures or predefined rules. These approaches are useful for known attacks, but they can struggle against modified, noisy, or previously unseen attack patterns.

Machine learning-based IDS systems address this limitation by learning behavioral differences between normal and malicious traffic. Instead of only matching signatures, an ML model can learn patterns in packet volume, timing, packet length, TCP flag behavior, retransmissions, and protocol type. This project uses this idea to build a supervised binary classifier for intrusion detection.

1.2 Problem Statement

The project solves the problem of automatically detecting malicious network behavior from captured traffic. Raw packet captures are too low-level for direct classification, so the system converts packet-level information into structured flow-based features. The final goal is to classify each observation window as Normal or Attack while minimizing both missed attacks and false alarms.

1.3 Objectives

- Create a complete supervised ML pipeline for intrusion detection.
- Transform raw network traffic into flow-based numerical features.
- Clean, normalize, and reduce the dataset before model training.
- Implement and compare Logistic Regression, Decision Tree, and Neural Network models.
- Evaluate all models using accuracy, precision, recall, F1-score, ROC-AUC, FPR, and FNR.
- Analyze overfitting, model stability, and generalization under two split ratios.
- Develop an interactive notebook interface for testing new traffic samples.

2. Dataset Collection and Preprocessing

2.1 Dataset Collection

The dataset was created from raw network traffic captured in PCAP/PCAPNG format. The project followed a controlled lab setup in which both normal and malicious activities were generated and then recorded. Wireshark/Tshark was used for packet capture and field extraction.

Normal traffic included Google Meet/WebRTC, HTTPS/TLS, QUIC/HTTP3, browsing, and streaming activity. Malicious traffic included controlled SYN flood, UDP flood, ICMP flood, SYN port scan, UDP scan, and aggressive scan scenarios. The attack traffic was generated only inside the isolated lab environment for academic experimentation.

The packet capture was converted into a structured CSV by extracting fields such as packet number, relative time, protocol, frame length, TCP/UDP source and destination ports, TCP flags, TCP window size, retransmission indicators, and duplicate ACK indicators. Packet rows were then aggregated into overlapping 5-second observation windows with 50% overlap.

2.2 Dataset Description

Property	Value
Dataset Type	Flow-based intrusion detection dataset
Total Samples	11,051
Original Columns	26 including Label
Original Predictors	25
Final Training Predictors	9 after leakage and correlation reduction
Classification Type	Binary classification
Classes	0 = Normal, 1 = Malicious
Window Size	5 seconds
Window Overlap	50%
Raw Class Distribution	Normal=5,211; Malicious=5,840
After 5% Label Flip in M3	Normal=5,267; Attack=5,784

The following table lists the features present in the prepared dataset and summarizes their purpose. The final implementation notebook used a reduced predictor set after dropping leakage-prone and highly correlated columns.

Feature	Description
total_packets	Number of packets observed in the flow/window. Used in final model.
total_bytes	Total bytes transferred in the window; removed later due to high correlation.
flow_duration_sec	Duration of the captured flow or traffic window in seconds. Used in final model.
bytes_per_sec	Byte rate, derived from total bytes and duration; removed due to correlation.
pkts_per_sec	Packet rate, derived from packet count and duration; removed due to correlation.
pkt_len_mean	Average packet length inside the window. Used in final model.
pkt_len_std	Variation in packet length; strong label correlation, removed during final correlation pruning.
pkt_len_min	Minimum packet length in the window. Used in final model.
pkt_len_max	Maximum packet length in the window; removed due to correlation with packet statistics.
iat_mean	Mean inter-arrival time between packets.
iat_std	Standard deviation of inter-arrival time.
iat_min	Minimum inter-arrival time; low values can indicate bursts. Used in final model.
iat_max	Maximum inter-arrival time in the window.
syn_count	Count of TCP SYN flags; removed in final correlation pruning.
ack_count	Count of TCP ACK flags; removed in final correlation pruning.
fin_count	Count of TCP FIN flags, reflecting connection teardown behavior. Used in final model.
rst_count	Count of TCP reset flags; removed in final correlation pruning.
syn_ack_ratio	Ratio between SYN and ACK behavior; treated as leakage-prone and dropped.
ttl_mean	Mean time-to-live value, describing routing/host behavior. Used in final model.
window_size_mean	Mean TCP window size; treated as leakage/redundancy-prone and dropped.
retransmission_count	Number of retransmissions observed in the window. Used in final model.
proto_tcp	Binary indicator for TCP traffic; removed in final correlation pruning.
proto_udp	Binary indicator for UDP traffic; removed in final correlation pruning.
proto_icmp	Binary indicator for ICMP traffic. Used in final model.
proto_other	Binary indicator for other protocols; dropped as low-value/leakage-prone.
Label	Target class: 0 = Normal traffic, 1 = Malicious traffic.

2.3 Data Cleaning

- Missing values: 0 missing values were found in the dataset.
- Duplicate handling: the implementation reported 0 duplicate records.
- Invalid entries: non-numeric and invalid packet fields were converted safely to numeric values and filled where necessary.

- Leakage/redundancy removal: proto_other, syn_ack_ratio, and window_size_mean were dropped before final training.
- Highly correlated columns: 13 columns were removed using an absolute correlation threshold of 0.85.

After final cleaning in ML_FINAL_M3, the training dataframe shape became 11,051 rows $\times$ 10 columns, consisting of 9 predictors plus Label.

2.4 Feature Engineering & Normalization

Packet-level rows were converted into window-level behavioral observations. Engineered features measured traffic volume, packet lengths, inter-arrival timing, TCP flags, retransmissions, TTL behavior, and protocol type. The background notebook used outlier clipping and RobustScaler during preprocessing, while the final implementation used StandardScaler after final feature selection for the train, validation, and test matrices.

- Scaling: continuous values were standardized for model training.
- Normalization: outlier-sensitive values were treated through clipping/log transformation in the preprocessing stage.
- Encoding: protocol identifiers were converted into binary protocol indicators such as proto_tcp, proto_udp, proto_icmp, and proto_other.
- Noise simulation: the final notebook injected MI-proportional Gaussian noise plus a 5% label flip to make generalization more realistic.

2.5 Exploratory Data Analysis

EDA was performed to validate class balance, data quality, feature correlation, feature distributions, skewness, redundancy, and separability between normal and attack traffic.



Figure 1. Class distribution and class percentage split.

The target class distribution graph shows that the dataset is relatively balanced, containing 5,211 normal traffic samples and 5,840 attack traffic samples. The pie chart further confirms that attack traffic represents 52.8% of the dataset, while normal traffic accounts for 47.2%. This balanced distribution helps reduce model bias toward a single class and supports more reliable intrusion detection performance.

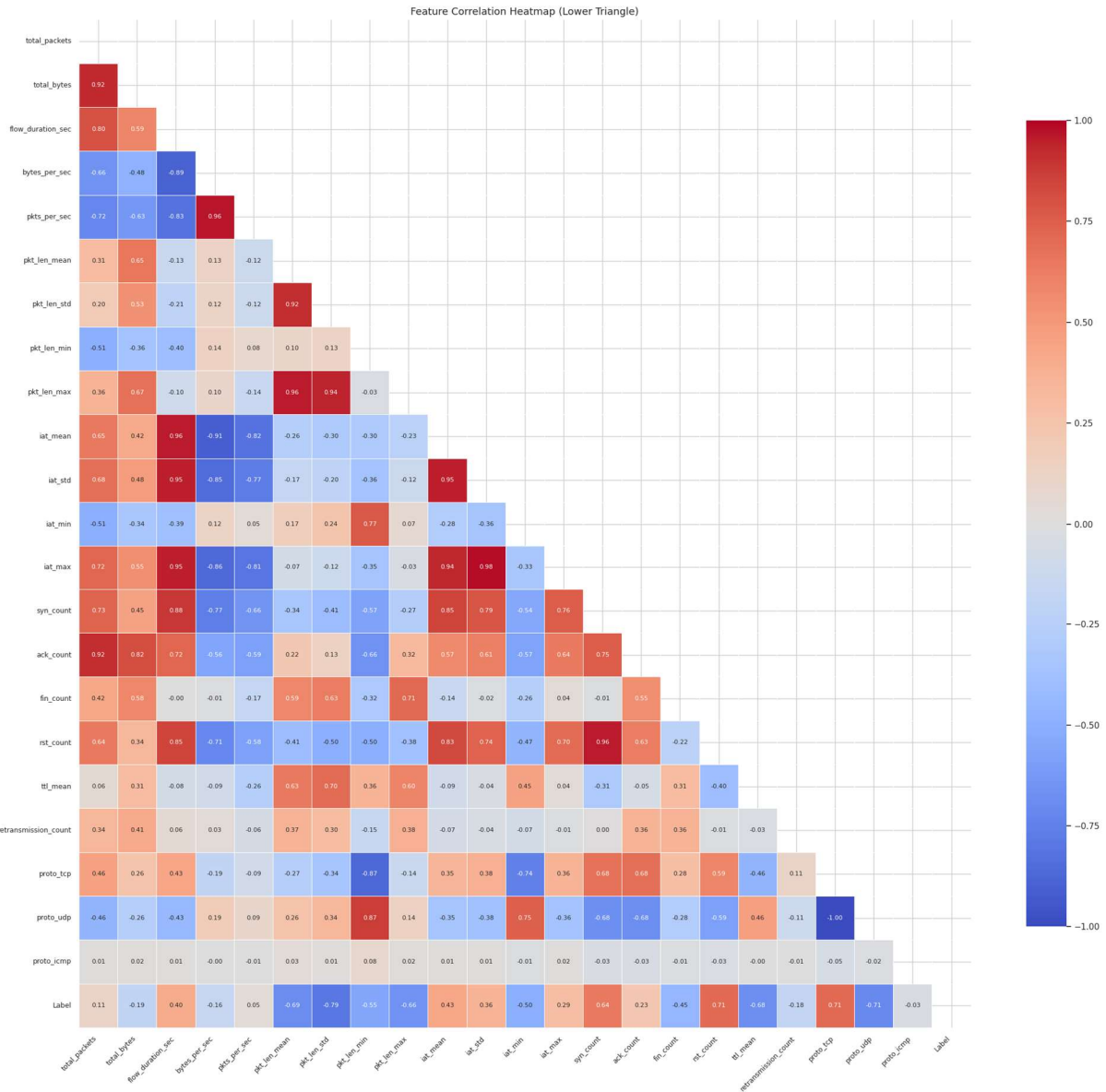


Figure 2. Lower-triangle correlation heatmap for dataset features.

The feature correlation heatmap illustrates the relationships between network traffic features used in the intrusion detection dataset. Strong positive correlations were observed among traffic-volume-related features such as total packets, total bytes, and packet rates, indicating that these variables increase together during high network activity. Certain timing-related features also showed high correlation, while some protocol-specific features exhibited negative correlations. The heatmap helps identify feature dependencies, multicollinearity, and important patterns within the dataset, supporting effective feature selection and model analysis.

Feature Distributions — Normal vs Attack Traffic

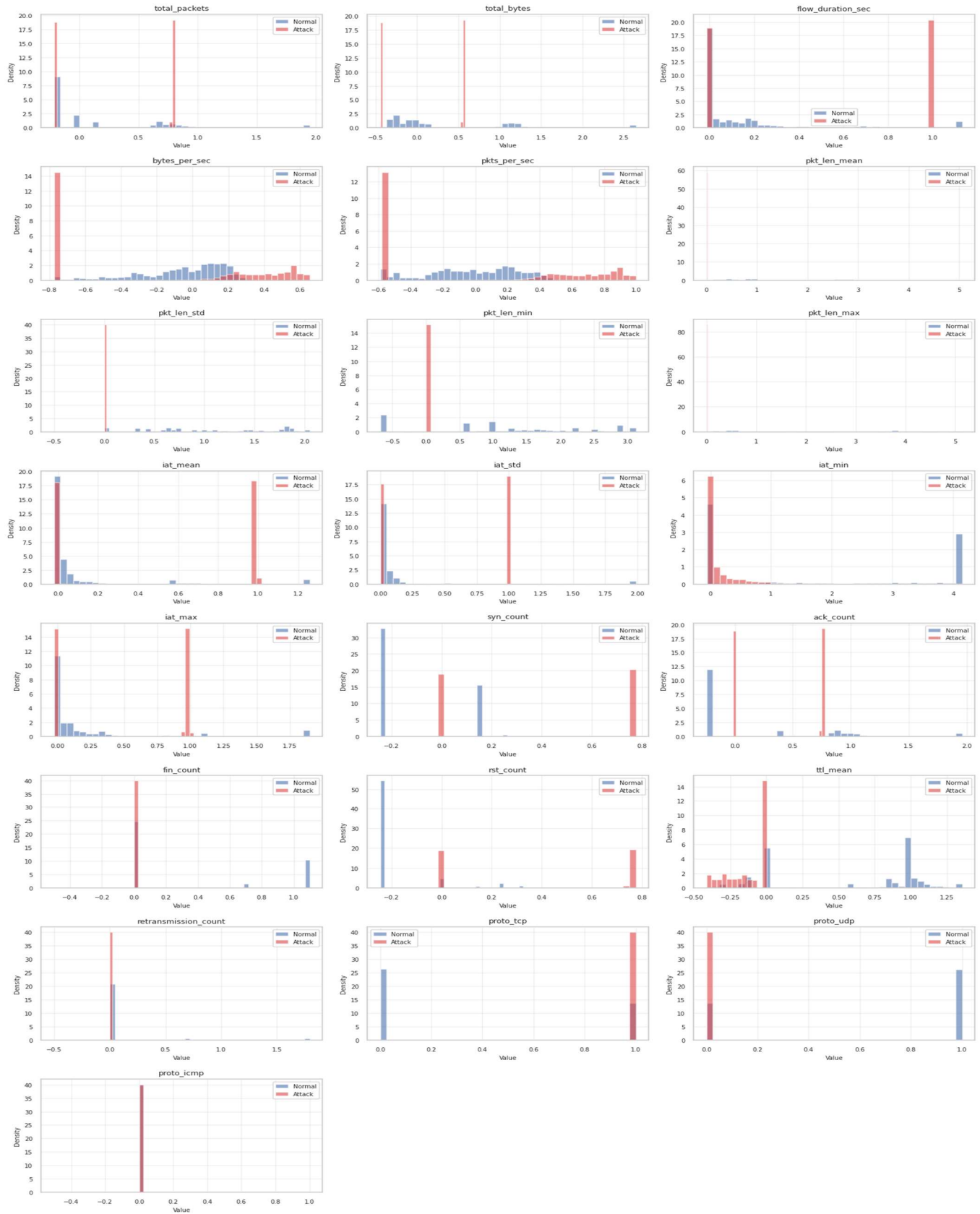


Figure 3. Feature distribution plots comparing Normal and Attack traffic.

The feature distribution plots compare the behavior of normal and attack network traffic across multiple extracted features. Several features, such as packet rates, inter-arrival times, and TCP flag counts, show noticeable differences between normal and malicious traffic patterns, indicating their importance for intrusion detection. Attack traffic generally exhibits more concentrated and distinct distributions, while normal traffic appears more dispersed across feature ranges. These visualizations help identify discriminative features and demonstrate that the dataset contains meaningful statistical differences suitable for supervised machine learning classification.

The EDA findings showed that the dataset was clean and almost balanced. Strong class-related features included packet length statistics, TCP protocol behavior, reset/SYN-related behavior, TTL mean, and timing features. Attack traffic showed tighter and more concentrated distributions, while normal traffic was more diverse.

3. Machine Learning Models

3.1 Logistic Regression Models

Logistic Regression was used as an interpretable and efficient baseline. Because it outputs probabilities, it is useful for threshold-based IDS decisions and FPR/FNR tuning. All LR variants used `class_weight="balanced"` and `max_iter=5000`.

Variant	Penalty	Main Parameters	Observation
LR-L1	L1 (LASSO)	C=0.1	Sparse coefficients and automatic feature selection.
LR-L2	L2 (Ridge)	C=0.1	Shrinks all weights smoothly for stable baseline performance.
LR-ElasticNet	ElasticNet	C=0.05, l1_ratio=0.6	Combines L1 sparsity and L2 stability.

3.1.1 LR-L1

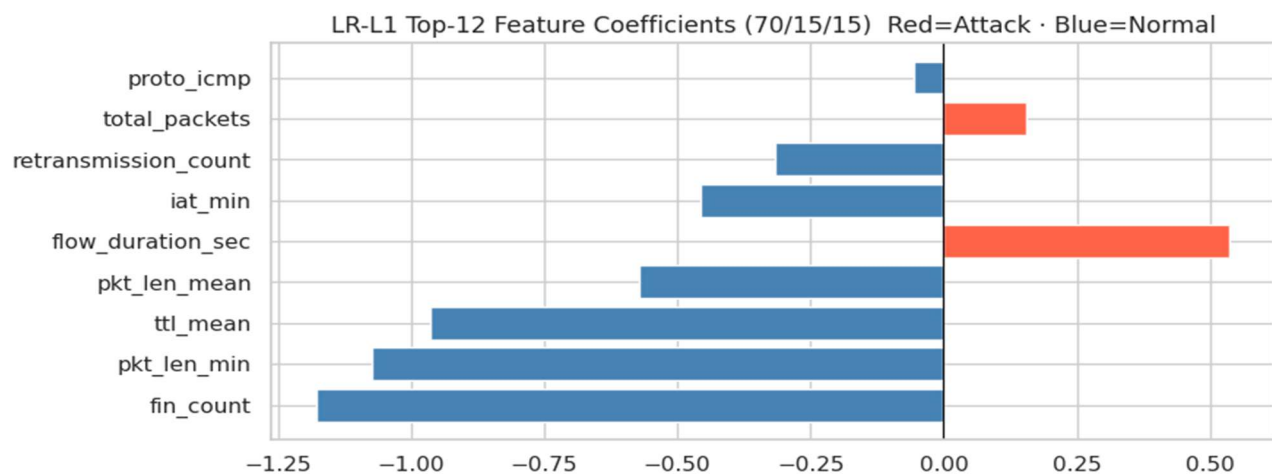
LR-L1 used LASSO regularization with C=0.1 and liblinear. It produced sparse coefficients and supported direct feature selection.

3.1.2 LR-L2

LR-L2 used Ridge regularization with C=0.1 and lbfgs. It kept all features while shrinking coefficients uniformly, producing stable results.

3.1.3 LR-ElasticNet

LR-ElasticNet used C=0.05, saga solver, and l1_ratio=0.6. It combined sparse and smooth regularization effects.



3.2 Decision Tree Models

Decision Trees were used because they produce rule-like decision paths and can model non-linear boundaries. The three variants were designed to reveal underfitting, balanced generalization, and overfitting behavior.

Variant	Depth	Criterion	Observation
DT-Shallow	max_depth=3	Gini	Highly interpretable but tends to underfit.
DT-Balanced	max_depth=7	Entropy	Best bias-variance balance among tree variants.
DT-Deep	None	Gini	Learns all split patterns, but overfits strongly.

3.2.1 DT-Shallow

DT-Shallow used max_depth=3. It was highly interpretable, but its limited depth reduced its ability to capture complex attack patterns.

3.2.2 DT-Balanced

DT-Balanced used max_depth=7 and entropy. It achieved the best tradeoff among tree models with consistent validation and test performance.

3.2.3 DT-Deep

DT-Deep had no depth limit. It reached perfect training performance but showed large train-validation gaps, confirming overfitting.

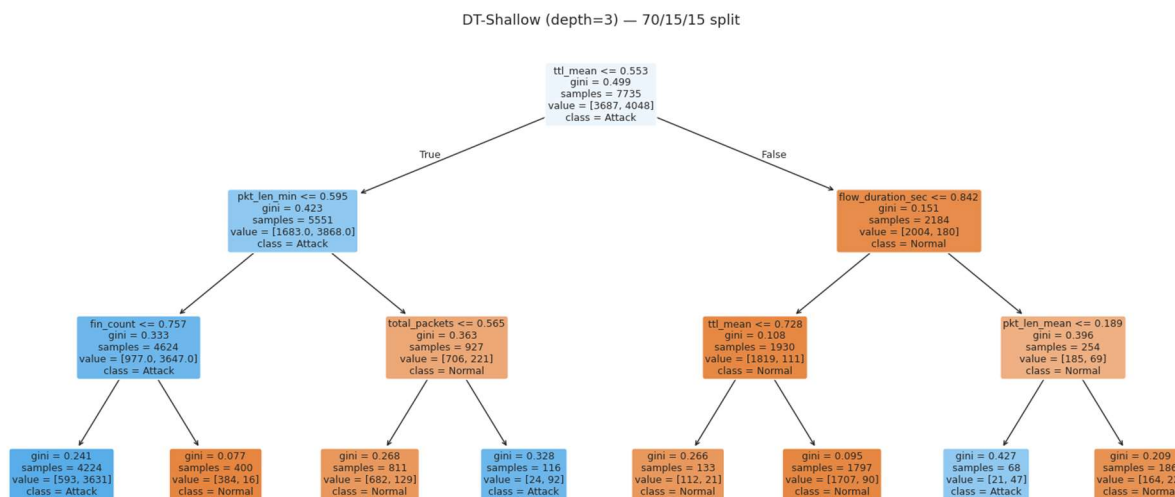


Figure 5. Shallow Decision Tree visualization.

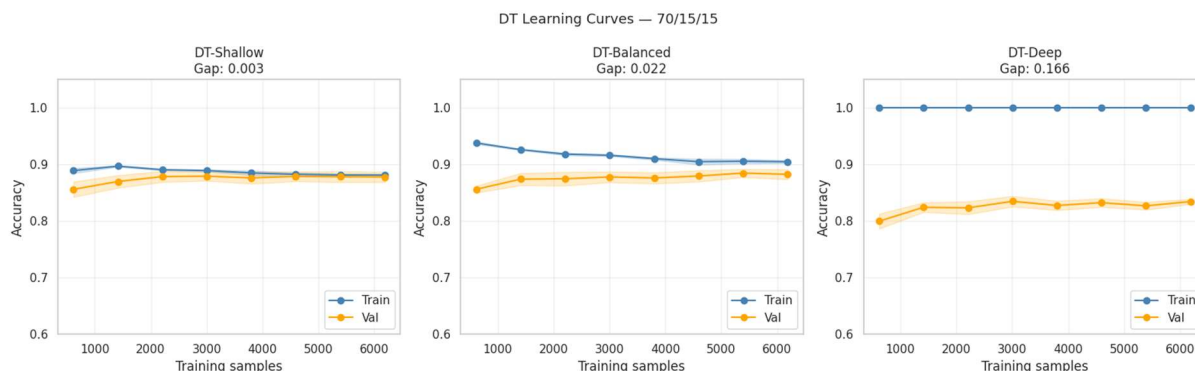


Figure 6. Decision Tree learning curves showing depth effects.

3.3 Neural Network Models

Neural Networks were used to model non-linear traffic behavior. All NN variants used dense layers, ReLU hidden activations, sigmoid output, Adam optimizer, binary cross-entropy loss, dropout, regularization, batch size 32, and early stopping on validation loss.

Variant	Architecture	Regularization	Observation
NN-Conservative	$16 > 8 > 1$	$L2=1e-2$; Dropout 0.6/0.5	Small and heavily regularized.
NN-Balanced	$32 > 16 > 1$	$L2=1e-3$; Dropout 0.5/0.4	Moderate capacity with stable learning.
NN-Aggressive	$64 > 32 > 16 > 1$	$L1=1e-4$; Dropout 0.3/0.2	Highest capacity; best final F1.

3.3.1 NN-Conservative

NN-Conservative used a small $16 > 8 > 1$ architecture with strong L2 regularization and high dropout. It emphasized generalization over capacity.

3.3.2 NN-Balanced

NN-Balanced used $32 > 16 > 1$ with moderate L2 regularization and dropout. It performed strongly while remaining stable.

3.3.3 NN-Aggressive

NN-Aggressive used $64 > 32 > 16 > 1$ with lighter dropout and L1 regularization. It had the highest capacity and achieved the best overall F1-score.

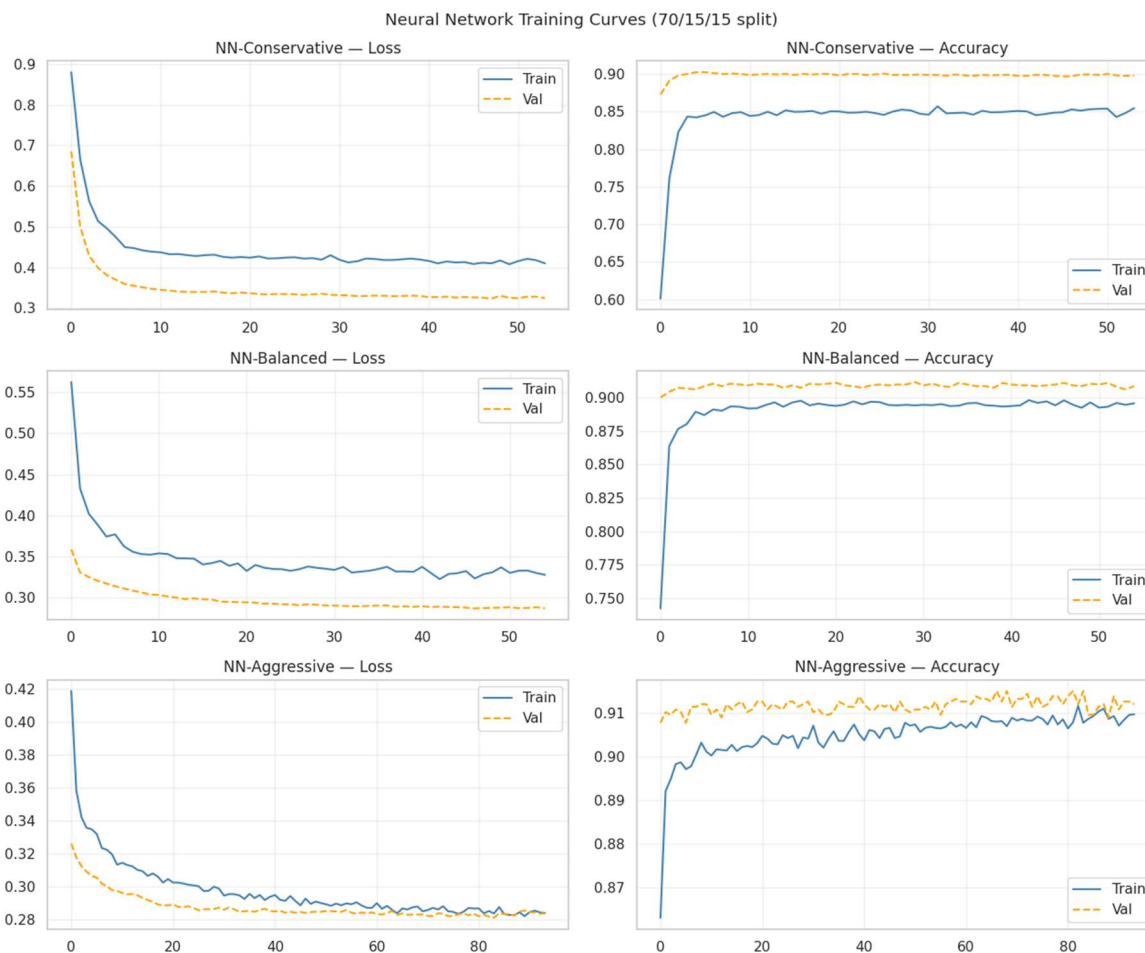


Figure 7. Neural Network training curves for loss and accuracy.

4. Training and Evaluation

4.1 Data Split Ratios

The final implementation evaluated every model under two stratified split ratios. Stratification preserved class balance across training, validation, and test subsets.

Split	Training Samples	Validation Samples	Test Samples	Purpose
70/15/15	7,735	1,658	1,658	Larger validation/test sets for stable evaluation.
80/10/10	8,840	1,105	1,106	More training data for capacity-heavy models.

4.2 Evaluation Metrics

Metric	IDS Interpretation
Accuracy	Overall proportion of correct predictions.
Precision	Among predicted attacks, how many were truly attacks.
Recall	Among real attacks, how many were detected.
F1-score	Harmonic mean of precision and recall; key IDS metric.
ROC-AUC	Discrimination strength across thresholds.
FPR	False Positive Rate: normal traffic incorrectly flagged as attack.
FNR	False Negative Rate: attack traffic incorrectly missed as normal.

For intrusion detection, F1-score and ROC-AUC were treated as the most important summary metrics because the system must balance missed attacks with false alarms.

5. Results and Comparative Analysis

5.1 Logistic Regression Results

Model	Split	Acc	Prec	Recall	F1	AUC	FPR	FNR
LR-L1	70/15/15	0.8739	0.8673	0.8963	0.8816	0.9198	0.1506	0.1037
LR-L2	70/15/15	0.8752	0.8701	0.8952	0.8825	0.9197	0.1468	0.1048
LR-ElasticNet	70/15/15	0.8758	0.8702	0.8963	0.8831	0.9199	0.1468	0.1037
LR-L1	80/10/10	0.8906	0.8742	0.9240	0.8984	0.9354	0.1461	0.0760
LR-L2	80/10/10	0.8906	0.8742	0.9240	0.8984	0.9353	0.1461	0.0760
LR-ElasticNet	80/10/10	0.8906	0.8742	0.9240	0.8984	0.9354	0.1461	0.0760

LR models were stable across splits. The 80/10/10 split improved F1 to 0.8984 for all LR variants, suggesting that the additional training data helped the linear decision boundary. Cross-validation F1 remained around 0.8955-0.8957 with low standard deviation, confirming stability.

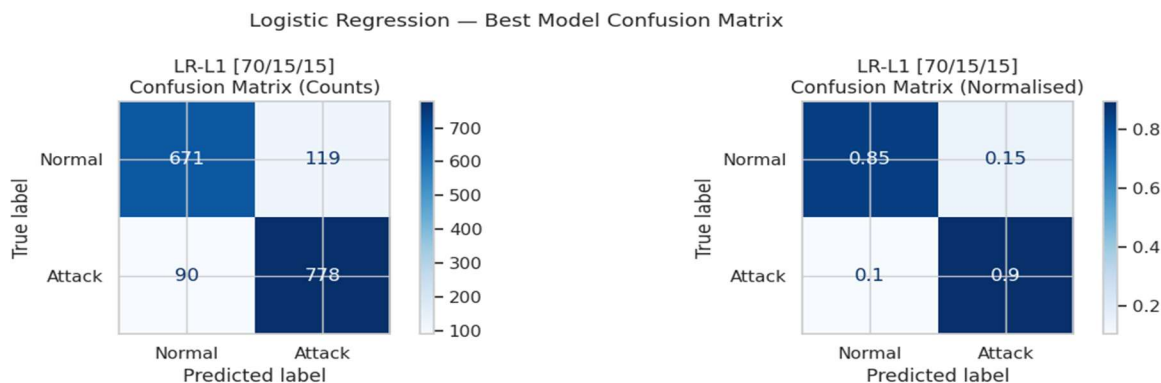


Figure 8. Best Logistic Regression confusion matrix.

Logistic Regression — ROC & PR Curves

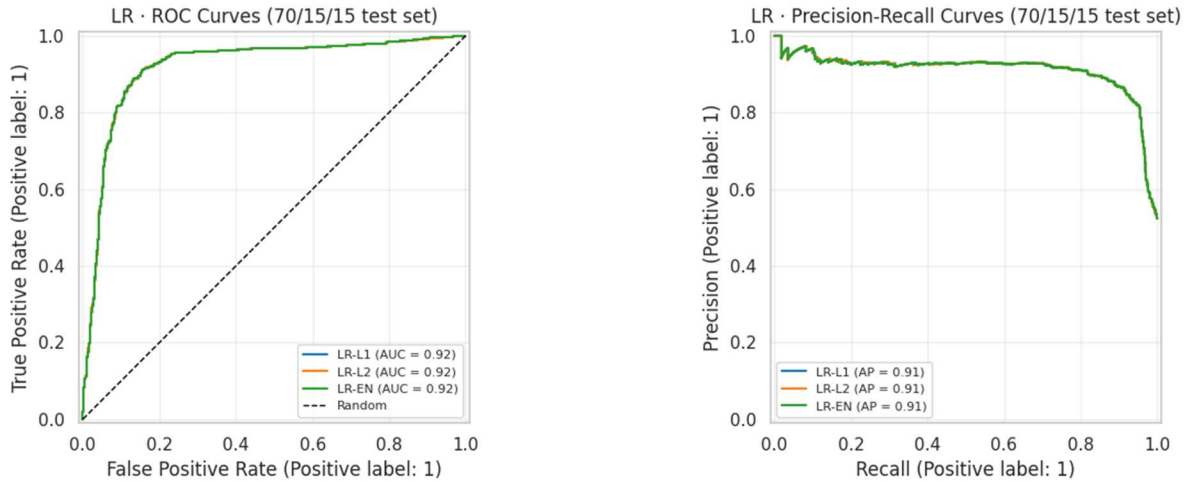


Figure 9. Logistic Regression ROC and Precision-Recall curves.

5.2 Decision Tree Results

Model	Split	Acc	Prec	Recall	F1	AUC	FPR	FNR
DT-Shallow	70/15/15	0.8607	0.8496	0.8917	0.8702	0.8801	0.1734	0.1083
DT-Balanced	70/15/15	0.8745	0.8548	0.9159	0.8843	0.9042	0.1709	0.0841
DT-Deep	70/15/15	0.8070	0.8239	0.8030	0.8133	0.8072	0.1886	0.1970
DT-Shallow	80/10/10	0.8590	0.8384	0.9050	0.8704	0.8786	0.1917	0.0950
DT-Balanced	80/10/10	0.8734	0.8501	0.9206	0.8839	0.9185	0.1784	0.0794
DT-Deep	80/10/10	0.8255	0.8362	0.8290	0.8326	0.8253	0.1784	0.1710

DT-Shallow underfit because depth 3 limited the number of decision rules. DT-Balanced performed best among tree models, reaching 0.8843 F1 under 70/15/15. DT-Deep memorized training data and suffered poor validation/test generalization.

Decision Tree — Best Model Confusion Matrix

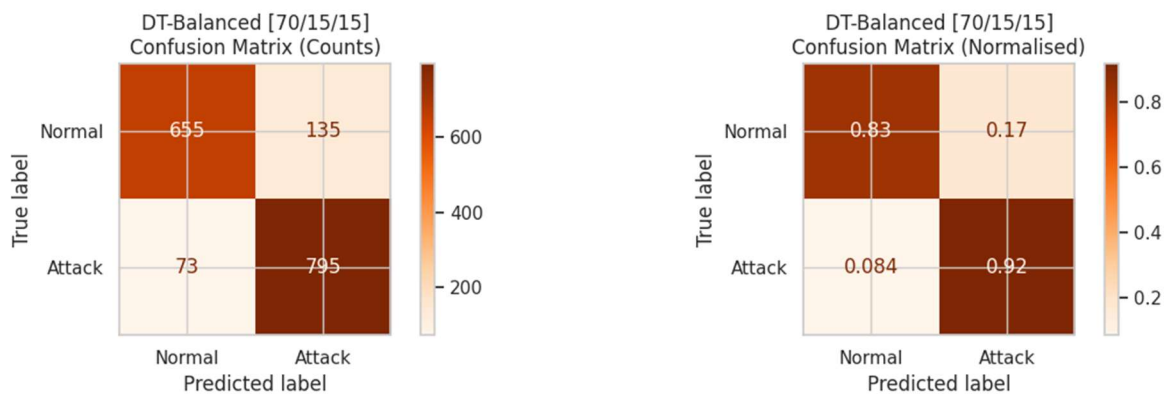


Figure 10. Best Decision Tree confusion matrix.

Decision Tree — ROC & PR Curves

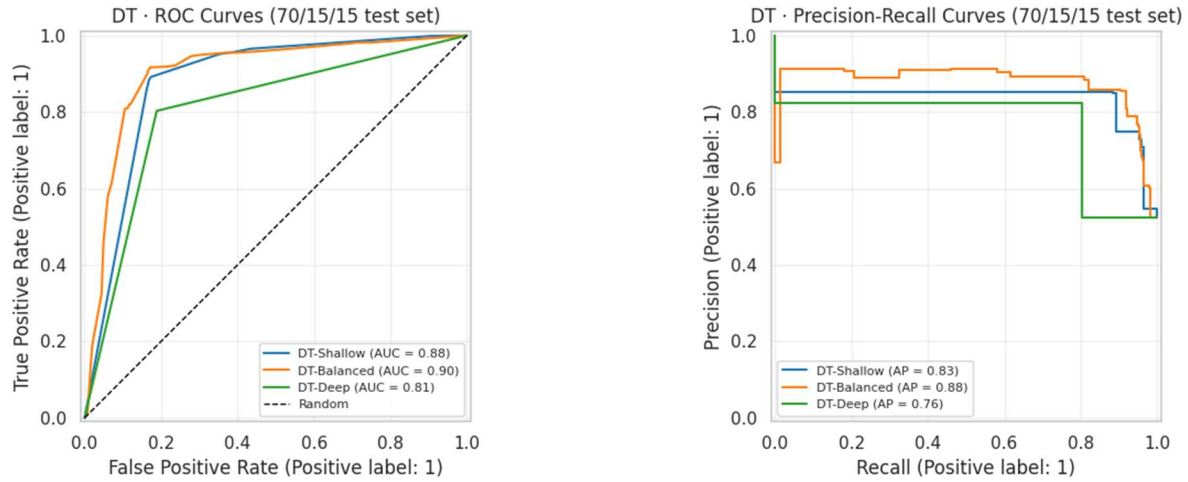


Figure 11. Decision Tree ROC and Precision-Recall curves.

5.3 Neural Network Results

Model	Split	Acc	Prec	Recall	F1	AUC	FPR	FNR
NN-Conservative	70/15/15	0.8758	0.8744	0.8906	0.8824	0.9218	0.1405	0.1094
NN-Balanced	70/15/15	0.8896	0.8879	0.9032	0.8955	0.9277	0.1253	0.0968
NN-Aggressive	70/15/15	0.8932	0.8985	0.8975	0.8980	0.9298	0.1114	0.1025
NN-Conservative	80/10/10	0.8897	0.8789	0.9154	0.8968	0.9360	0.1385	0.0846
NN-Balanced	80/10/10	0.8996	0.8926	0.9188	0.9055	0.9404	0.1214	0.0812
NN-Aggressive	80/10/10	0.9033	0.8933	0.9257	0.9092	0.9377	0.1214	0.0743

Neural Networks achieved the best overall performance. The aggressive architecture performed best because its larger capacity captured non-linear traffic patterns while dropout, L1 regularization, and early stopping controlled overfitting.

Neural Network — Best Model Confusion Matrix

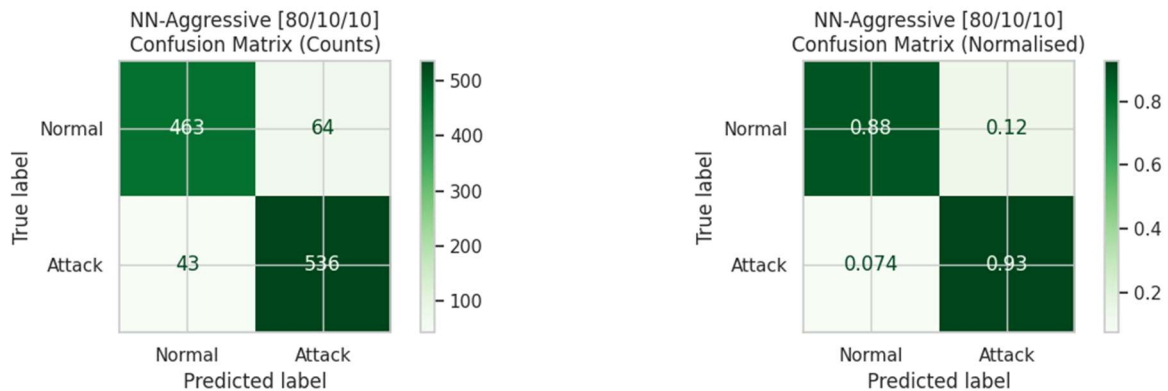


Figure 12. Best Neural Network confusion matrix.

Neural Network — ROC & PR Curves

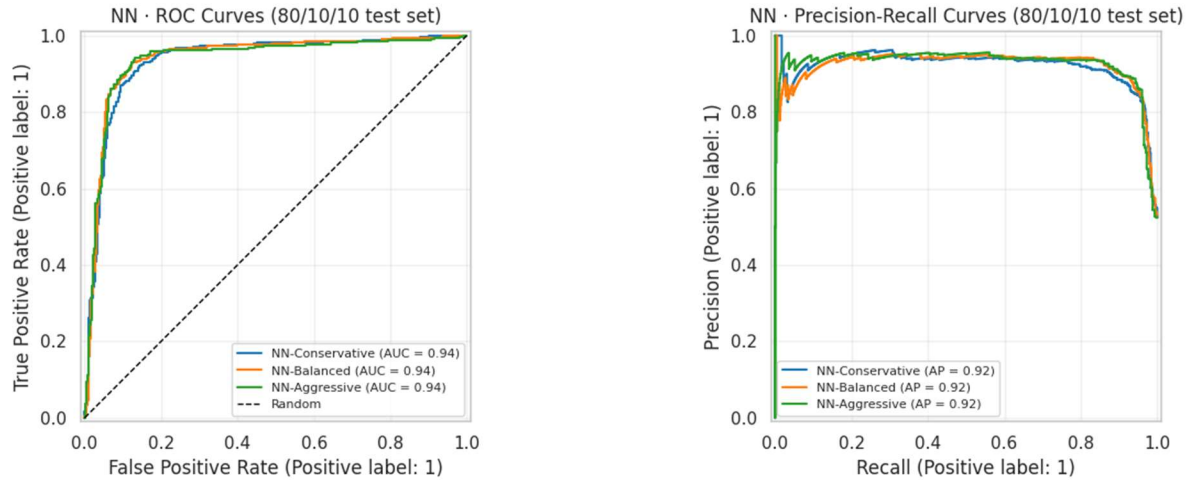


Figure 13. Neural Network ROC and Precision-Recall curves.

5.4 Comparative Performance Table

Model	Split	Accuracy	F1	ROC-AUC	FPR	FNR
NN-Aggressive	80/10/10	0.9033	0.9092	0.9377	0.1214	0.0743
NN-Balanced	80/10/10	0.8996	0.9055	0.9404	0.1214	0.0812
LR-ElasticNet	80/10/10	0.8906	0.8984	0.9354	0.1461	0.0760
LR-L2	80/10/10	0.8906	0.8984	0.9353	0.1461	0.0760
LR-L1	80/10/10	0.8906	0.8984	0.9354	0.1461	0.0760
NN-Aggressive	70/15/15	0.8932	0.8980	0.9298	0.1114	0.1025
NN-Conservative	80/10/10	0.8897	0.8968	0.9360	0.1385	0.0846
NN-Balanced	70/15/15	0.8896	0.8955	0.9277	0.1253	0.0968
DT-Balanced	70/15/15	0.8745	0.8843	0.9042	0.1709	0.0841
DT-Balanced	80/10/10	0.8734	0.8839	0.9185	0.1784	0.0794
LR-ElasticNet	70/15/15	0.8758	0.8831	0.9199	0.1468	0.1037
LR-L2	70/15/15	0.8752	0.8825	0.9197	0.1468	0.1048
NN-Conservative	70/15/15	0.8758	0.8824	0.9218	0.1405	0.1094
LR-L1	70/15/15	0.8739	0.8816	0.9198	0.1506	0.1037
DT-Shallow	80/10/10	0.8590	0.8704	0.8786	0.1917	0.0950
DT-Shallow	70/15/15	0.8607	0.8702	0.8801	0.1734	0.1083
DT-Deep	80/10/10	0.8255	0.8326	0.8253	0.1784	0.1710
DT-Deep	70/15/15	0.8070	0.8133	0.8072	0.1886	0.1970

5.5 Comparative Graphs

Performance Comparison — All 18 Models

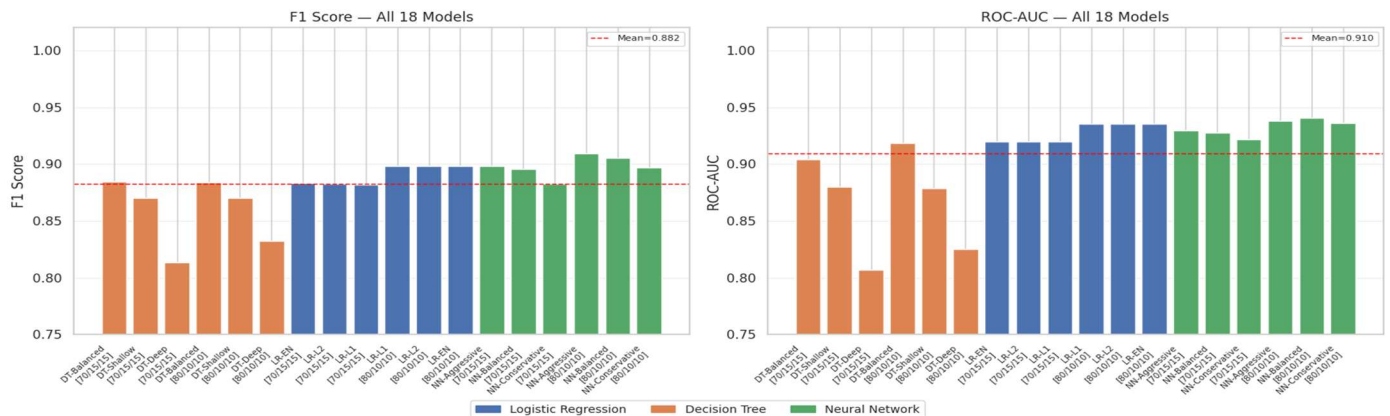


Figure 14. F1 and ROC-AUC comparison for all 18 models.



Figure 15. All-model performance heatmap.

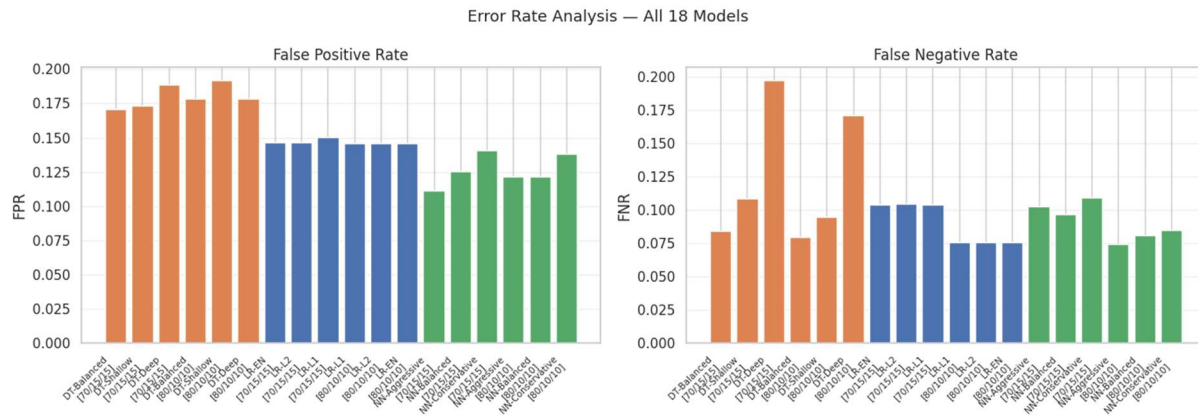


Figure 16. False positive and false negative rate comparison.

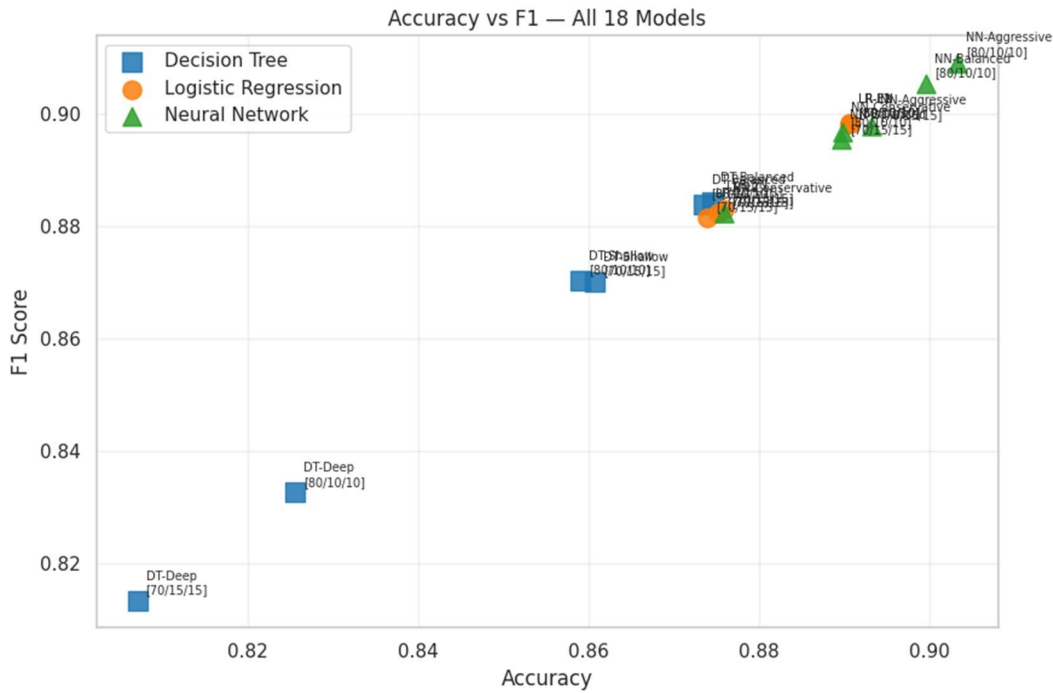


Figure 17. Accuracy versus F1 scatter plot.

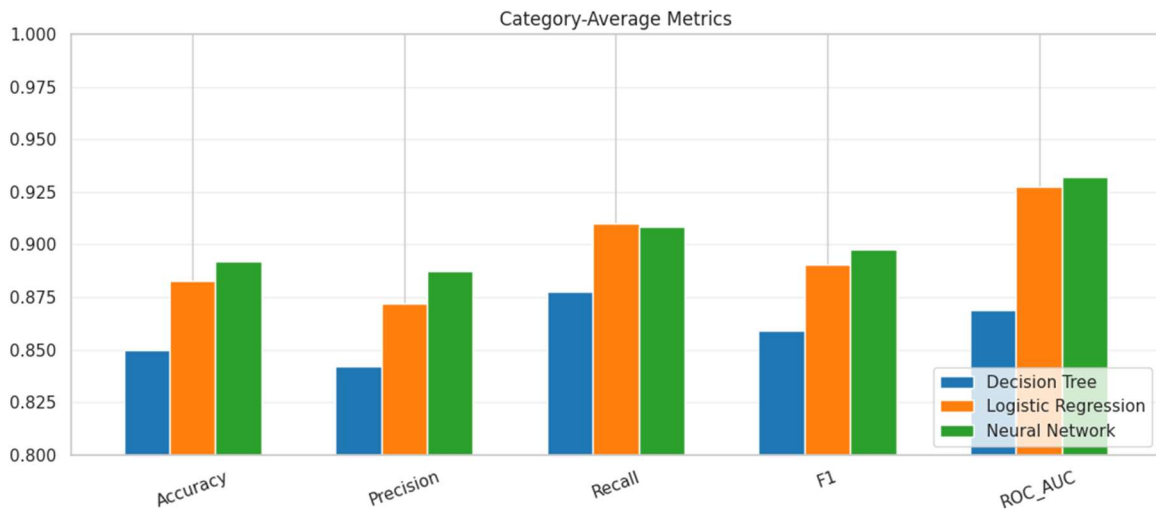


Figure 18. Category-average metrics across model families

The comparative graphs show that Neural Network models achieved the best overall performance across most evaluation metrics, particularly in F1-score and ROC-AUC, with the NN-Aggressive model using the 80/10/10 split producing the strongest results. Logistic Regression models demonstrated highly stable and consistent performance with strong generalization across both split ratios, while Decision Tree models showed greater variation, where shallow and balanced trees performed reasonably well but deep trees suffered from overfitting. The heatmap and category-average graphs further confirm that Neural Networks and Logistic Regression achieved higher Accuracy, Precision, Recall, F1-score, and ROC-AUC values with lower False Negative Rates compared to Decision Trees. The error-rate analysis also indicates that Neural Networks maintained a better balance between false positives and false negatives, making them more effective for intrusion detection tasks. Overall, the graphs demonstrate that Neural Networks provided the most reliable classification performance, while Logistic Regression offered stable and interpretable results across different experimental settings.

5.6 Split Ratio Analysis

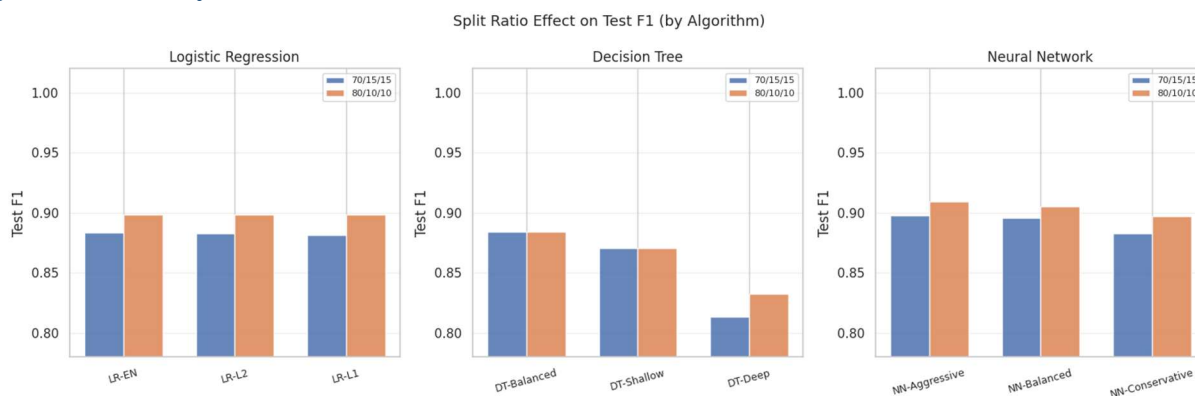


Figure 19. Split ratio effect on test F1 by algorithm family.

The 80/10/10 split improved Logistic Regression and Neural Network performance because it provided more training samples. Decision Tree Balanced remained almost unchanged across both splits, while DT-Deep continued to show overfitting. The split analysis confirms that the best model benefited from additional training data without a harmful increase in the generalization gap.

Category	Accuracy	Precision	Recall	F1	ROC-AUC	FPR	FNR
Decision Tree	0.8500	0.8422	0.8775	0.8591	0.8690	0.1802	0.1225
Logistic Regression	0.8828	0.8717	0.9100	0.8904	0.9276	0.1471	0.0900
Neural Network	0.8919	0.8876	0.9085	0.8979	0.9322	0.1264	0.0915



Figure 20. Training time comparison for all 18 models.

6. Overfitting and Generalization Analysis

Generalization was evaluated using the Train F1 minus Validation F1 gap. A gap above 0.05 was treated as a warning sign of overfitting. Most models stayed below the threshold, but both DT-Deep runs exceeded it by a large margin.

Model	Train F1	Validation F1	Gap
DT-Deep [70/15/15]	1.0000	0.8476	0.1524
DT-Deep [80/10/10]	1.0000	0.8236	0.1764

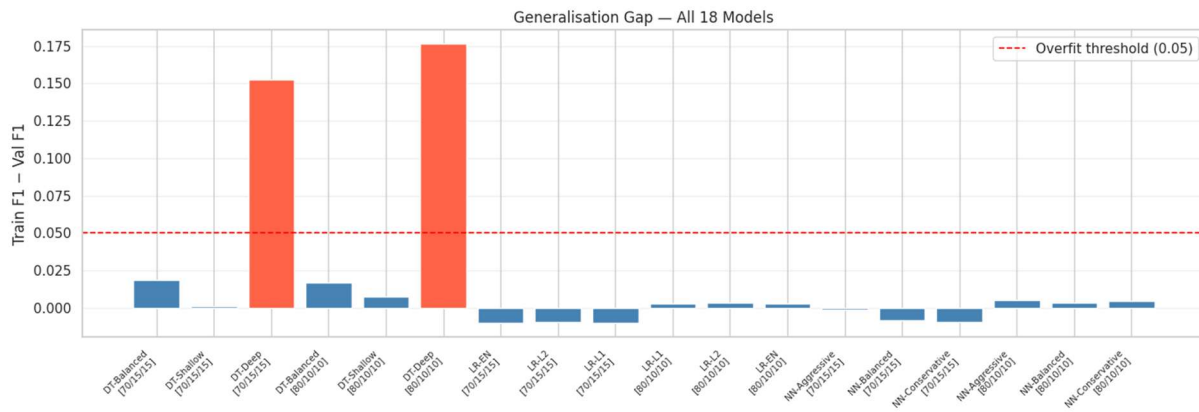


Figure 21. Generalization gap across all 18 models.

- DT-Deep overfitting: achieved Train F1 = 1.0000 but poor validation/test performance.
- LR stability: small train-validation gaps and strong cross-validation stability.
- NN regularization effectiveness: dropout, L1/L2 penalties, and early stopping kept gaps below 0.05 while improving F1.

7. Interactive Notebook Interface

The notebook includes an interactive prediction interface that asks the user to enter values for the final 9 model features. If the user presses Enter, the median training value is used as a default. The interface runs all 9 models trained under the 70/15/15 split and returns each model prediction, attack probability, normal probability, and a majority-vote ensemble result.

```
=====
IDS INTERACTIVE PREDICTION INTERFACE
(70/15/15 split - 9 models - with attack probability)
=====

Enter values for 9 features.
Press [Enter] to use median (shown in brackets).

total_packets           [default=-0.2300]:
flow_duration_sec       [default=-0.1720]:
pkt_len_mean            [default=-0.1518]:
pkt_len_min             [default=-0.1248]:
iat_min                 [default=-0.2311]:
fin_count               [default=-0.3327]:
ttl_mean                [default=-0.2368]:
retransmission_count    [default=-0.1539]:
proto_icmp              [default=-0.0325]:

=====
PREDICTION RESULTS
=====
```

Model	Prediction	Atk Prob	Nrm Prob
LR-L1	ATTACK	67.2%	32.8%
LR-L2	ATTACK	67.0%	33.0%
LR-EN	ATTACK	66.7%	33.3%
DT-Shallow	ATTACK	86.0%	14.0%
DT-Balanced	ATTACK	85.7%	14.3%
DT-Deep	ATTACK	100.0%	0.0%
NN-Conservative	ATTACK	74.2%	25.8%
NN-Balanced	ATTACK	73.0%	27.0%
NN-Aggressive	ATTACK	52.6%	47.4%

```
=====
ENSEMBLE VOTE (9/9 models):  ATTACK
Average Attack Probability : 74.7%
Average Normal Probability : 25.3%
=====
```

Automated batch testing was also added. This section tested attack examples, normal examples, and ambiguous edge cases. The ensemble correctly classified 5 out of 6 labelled test cases. The failed edge case was important because it revealed a feature masking limitation where packet-length values made an attack sample appear normal.

```

=====
BATCH TEST SUMMARY
=====
Test Case                               Exp    Pred    AtkProb  Verdict
-----
Attack 1 (SYN Flood)                   ATTACK ATTACK    73.9%  ✓ CORRECT
Attack 2 (Aggressive SYN)               ATTACK ATTACK    83.5%  ✓ CORRECT
Normal 1 (Baseline)                     Normal Normal     5.3%  ✓ CORRECT
Normal 2 (Long Session)                 Normal Normal    13.9%  ✓ CORRECT
Edge 1 (Attack disguised as Normal)     ATTACK Normal    11.8%  ✗ WRONG
Edge 2 (Normal looks suspicious)        Normal Normal    26.1%  ✓ CORRECT
Edge 3 (Fully ambiguous ~50%)           ??? Normal    24.4%  N/A (ambiguous)

Score: 5/6 correct (Edge 3 excluded – no ground truth)
=====

```

Figure 23. Automated batch testing summary.

8. Best Model Discussion

Item	Value
Best Model	NN-Aggressive
Split	80/10/10
Category	Neural Network
Accuracy	0.9033
Precision	0.8933
Recall	0.9257
F1-score	0.9092
ROC-AUC	0.9377
FPR	0.1214
FNR	0.0743

NN-Aggressive performed best because it had enough capacity to learn non-linear relationships among timing, packet-length, TCP behavior, TTL, retransmission, and protocol features. Its recall of 0.9257 is valuable in IDS because recall directly measures how many real attacks are detected.

The model also maintained a low generalization gap, showing that the additional layers did not simply memorize the training data. Dropout and L1 regularization reduced overfitting risk, while early stopping restored the best validation weights. Its main weakness is computational cost: Neural Networks required much longer training time than Logistic Regression and Decision Trees.

9. Limitations

- Dataset scope: traffic was generated in a controlled lab, so it may not fully represent all real enterprise network conditions.
- Attack diversity: only selected flood and scanning attacks were included; more attack families would improve robustness.
- Feature masking: one edge case showed that normal-looking packet-length features can hide malicious behavior.
- Computational overhead: Neural Networks achieved the best performance but required significantly longer training time.
- Static evaluation: the project evaluated offline datasets, not a real-time streaming IDS pipeline.
- Label noise: 5% label flipping improved realism but can also make boundary cases harder to interpret.

10. Future Improvements

- Deploy the model inside a real-time IDS pipeline that consumes live network flow features.
- Add more attack categories such as brute force, botnet, exfiltration, malware beaconing, and web attacks.
- Use ensemble learning methods such as Random Forest, XGBoost, LightGBM, and stacking ensembles.
- Apply online learning to adapt to changing network behavior over time.
- Experiment with transformer-based IDS models for sequence-aware packet/flow modeling.
- Add threshold tuning so administrators can choose a lower FNR or lower FPR operating point.
- Improve interpretability using SHAP/LIME and feature-level explanation dashboards.

11. Conclusion

This project successfully developed an end-to-end supervised machine learning system for intrusion detection. The workflow covered traffic collection, packet extraction, feature engineering, preprocessing, exploratory analysis, model implementation, training, evaluation, comparative analysis, and an interactive prediction interface.

The dataset contained 11,051 samples and was suitable for binary classification due to its near-balanced class distribution and strong behavioral features. EDA confirmed that timing, packet-length, TTL, retransmission, and protocol behavior features helped separate normal and malicious traffic.

Among the 18 trained models, NN-Aggressive under the 80/10/10 split achieved the best overall performance with F1-score 0.9092 and ROC-AUC 0.9377. Logistic Regression provided a stable and interpretable baseline, Decision Tree Balanced offered interpretable rules with reasonable performance, and DT-Deep demonstrated the risk of unconstrained model complexity. Overall, the results show that supervised machine learning can provide effective intrusion detection when raw packet captures are transformed into meaningful flow-level features.

12. References

1. Scikit-learn Developers. LogisticRegression documentation. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
2. Scikit-learn Developers. DecisionTreeClassifier documentation. <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>
3. Scikit-learn Developers. StandardScaler documentation. <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>
4. Scikit-learn Developers. Model evaluation and scoring documentation. https://scikit-learn.org/stable/modules/model_evaluation.html
5. TensorFlow. tf.keras.Sequential API documentation. https://www.tensorflow.org/api_docs/python/tf/keras/Sequential
6. TensorFlow. tf.keras.layers.Dense API documentation. https://www.tensorflow.org/api_docs/python/tf/keras/layers/Dense
7. TensorFlow. tf.keras.layers.Dropout API documentation. https://www.tensorflow.org/api_docs/python/tf/keras/layers/Dropout
8. TensorFlow. tf.keras.losses.BinaryCrossentropy API documentation. https://www.tensorflow.org/api_docs/python/tf/keras/losses/BinaryCrossentropy
9. Wireshark Foundation. tshark(1) Manual Page. <https://www.wireshark.org/docs/man-pages/tshark.html>
10. Nmap Project. Nmap Reference Guide. <https://nmap.org/book/man.html>
11. Divekar, A., et al. Benchmarking datasets for anomaly-based network intrusion detection. arXiv:1811.05372, 2018.
12. Pinto, A., et al. Survey on Intrusion Detection Systems Based on Machine Learning Techniques for the Protection of Critical Infrastructure. Sensors, 2023.
13. Abdallah, E. E., et al. Intrusion Detection Systems using Supervised Machine Learning Techniques: A Survey. Procedia Computer Science, 2022.

13. Generative AI Use Declaration

ChatGPT and Claude was used for debugging assistance, visualization suggestions, and report refinement. All implementation, experimentation, analysis, and final decisions were performed and verified.

Appendix

Appendix A: Final Predictor Set Used in ML_FINAL_M3

Final Predictor	Reason for Inclusion
total_packets	Captures traffic volume.
flow_duration_sec	Captures time span of a flow/window.
pkt_len_mean	Captures average packet size behavior.
pkt_len_min	Captures minimum packet size pattern.
iat_min	Captures burst speed and minimum packet spacing.
fin_count	Captures connection teardown behavior.
ttl_mean	Captures routing/host behavior pattern.
retransmission_count	Captures TCP retransmission behavior.
proto_icmp	Captures ICMP protocol activity.

Appendix B: Additional Test Case Insight

The automated batch testing showed strong performance on clear attack and normal samples, but it also revealed that disguised or ambiguous samples can still fail. This makes future threshold tuning, more diverse attack data, and ensemble models valuable improvements.

Appendix B: COLAB FILE LINKS FOR REFERENCE

https://colab.research.google.com/drive/1k7bhZmr5fhxDIi_Wr5iRU22zAHDGlt3h?usp=sharing

(MILESTONE 3 ALGORITHM IMPLEMENTATION)

https://colab.research.google.com/drive/1Ek-1So10x7a1Jvcqp9EOUa_AlON5xYTj?usp=sharing

(ADDITIONAL DATA ANALYSIS)